

InterviewEase : AI-powered interview assistance

Param Kothari

`param.kothari@spit.ac.in`

Bhartiya Vidya Bhavans' Sardar Patel Institute of Technology

Paras Mehta

Bhartiya Vidya Bhavans' Sardar Patel Institute of Technology

Srushti Patil

Bhartiya Vidya Bhavans' Sardar Patel Institute of Technology

Prof. Varsha Hole

Bhartiya Vidya Bhavans' Sardar Patel Institute of Technology

Research Article

Keywords: placement preparation, AI-driven, skill gap analysis, mock interview, gesture detection, speech emotion recognition, question generator

Posted Date: February 28th, 2024

DOI: <https://doi.org/10.21203/rs.3.rs-3964944/v1>

License:   This work is licensed under a Creative Commons Attribution 4.0 International License.

[Read Full License](#)

Additional Declarations: No competing interests reported.

InterviewEase : AI-powered interview assistance

Param Kothari^{1,2}, Paras Mehta^{1,3}, Srushti Patil^{1,4},
Prof. Varsha Hole^{1,5}

¹Sardar Patel Insititute of Technology, Mumbai, India.

Contributing authors: param.kothari@spit.ac.in; paras.mehta@spit.ac.in;
srushti.patil2@spit.ac.in; varsha_hole@spit.ac.in;

Abstract

InterviewEase addresses the need for automating the placement preparation process with a comprehensive AI-driven placement preparation platform, comprising three modules: (1) a ChatGPT-based skill gap analyzer recommending skill-enhancing projects based on the skill gap between resume and job requirements; (2) a mock interview platform utilizing gesture detection, speech emotion recognition, and answer similarity algorithms for comprehensive feedback on posture, speech, and answers; (3) a customizable question generator with adjustable difficulty levels. The mock interview module attains the following accuracy levels: 98% for body gesture detection, 82% for speech emotion recognition, and 92% for answer similarity comparison.

Keywords: placement preparation, AI-driven, skill gap analysis, mock interview, gesture detection, speech emotion recognition, question generator

1 Introduction

In the contemporary job market, where competition is fierce and industry demands are ever-evolving, the importance of meticulous job placement preparation cannot be overstated. Academic excellence, while valuable, is no longer the sole guarantee of success. A strategic alignment of skills, polished resume presentation, and adept interview performance have become imperative. Recognizing the intricacies of this transition, InterviewEase emerges as a pioneering platform poised to redefine the landscape of placement preparation.

Navigating the intricacies of job applications can be challenging, particularly when showcasing the right skills on a resume. Often, candidates miss out on vital skills

required for a specific job simply because they are unaware of them. The resume-skill gap analyzer module within InterviewEase aims to address this concern by comparing the skills listed on a resume with the requirements of the job role. This tool not only highlights any missing skills but also recommends relevant projects to bridge the identified gaps.

Some companies also take hiring rounds via video interviews and conference calls, where confident and well-defined body language is essential. However, existing interview preparation platforms often rely on text-based methods, requiring manual intervention and offering limited efficiency. While they may offer feedback solely on the correctness of answers, they often overlook the nuances of a candidate's gestures. The mock interview module is designed to equip students with the skills and confidence needed to thrive in real-world job interviews. It provides demo questions with concise answers, enabling students to focus on refining both their verbal and non-verbal communication skills.

While valuable resources for interview questions on topics like Object-Oriented Programming (OOPS), Data Structures and Algorithms (DSA), Database Management Systems (DBMS), Computer Communication and Networking (CCN), and more are scattered across the internet, the manual effort required to find questions of the desired difficulty level can be time-consuming. InterviewEase eliminates this by generating questions tailored to the user's preferred difficulty level.

In summary, the platform's streamlined approach comprehensively covers every aspect of the interview process, including resume shortlisting and enhancement, mock interviews, and automatic test question generation. This results in a simplified and accelerated placement preparation journey, effectively reducing the manual effort required for interview assessments, resource gathering, and the preparation of practice test sets.

2 Related Work

[1] discusses the utilization of Optical Character Recognition (OCR) and Natural Language Processing (NLP) techniques for resume parsing, employing tokenization and semantic analysis. This process generates structured resumes, and employee resumes are ranked based on the company's skill set requirements and the skills presented in the provided resumes.

[2] introduces a novel task of topic-based question generation, using the Amazon question/answer corpus1 (AQAD) [3] for training and testing. The authors set the number of layers of Long Short-Term Memory (LSTM) to 1 in both the encoder and decoder. Network parameters are updated with the Adam algorithm [4], using a learning rate of 0.0001. Additionally, a sentence classification model is employed. The paper proposes a new network architecture for the task, discussing its generalization ability. The system generated questions based on given topics, which proved relevant for assessments. Subsequently, the study delves into various areas to aid in the implementation of a planned interview bot.

[5] explains an interview bot utilizing machine learning to assess a person's competence level. The study employs an Indonesian language dataset, implementing NLP and statistical analyses for model construction. Bayesian inference is applied to update prior probabilities based on new data. The study includes preprocessing steps such as removing stopwords, stemming, executing part-of-speech (POS) analysis, and calculating Term Frequency. NLP technology is used for human interaction. The study shows high accuracy in various scenarios, serving as a foundation for developing an interview bot closer to professional interviews. Suggestions for enhancement include incorporating voice interaction and adapting the frequency, types, and depth of questions to match the psychological aspects of the interviewee.

[6] classifies a person's performance in an interview based on video analysis. The paper explains a system performing resume parsing and suggesting jobs based on user skills using deep learning. This includes extracting data from resumes and utilizing speech emotion recognition to gauge user emotions while answering. Facial emotion recognition is explained as part of evaluating the interviewee's emotions. Convolutional Neural Network (CNN) and Deep Neural Network (DNN) models are employed for these technologies, utilizing various behavioral cues and signals accessible through different routes to detect human affect.

[7] explores the connection between subjective perceptions and auditory nonverbal communication displayed by interviewees. The study investigates how interviewees' acoustic characteristics change due to various conversational subjects, aiming to extrapolate observed and self-assessed attitudes and body language. Findings indicate that topicality plays a key role in the statistics of acoustic characteristics.

[8] extracts audio, single video frames, and sequences of video frames from multimedia sources. Feature vectors are then extracted using OpenSMILE for audio and Inception-ResNet-v2 for static images. Emotion classification into seven categories (angry, disgust, fear, happy, sad, surprise, and neutral) is performed using Support Vector Machine (SVM) for audio, Inception-ResNet-v2 for static images, and the C3D and LSTM models for video sequences. The study notes challenges in training C3D models, resulting in overfitting and poor accuracy on the training dataset.

3 System Design

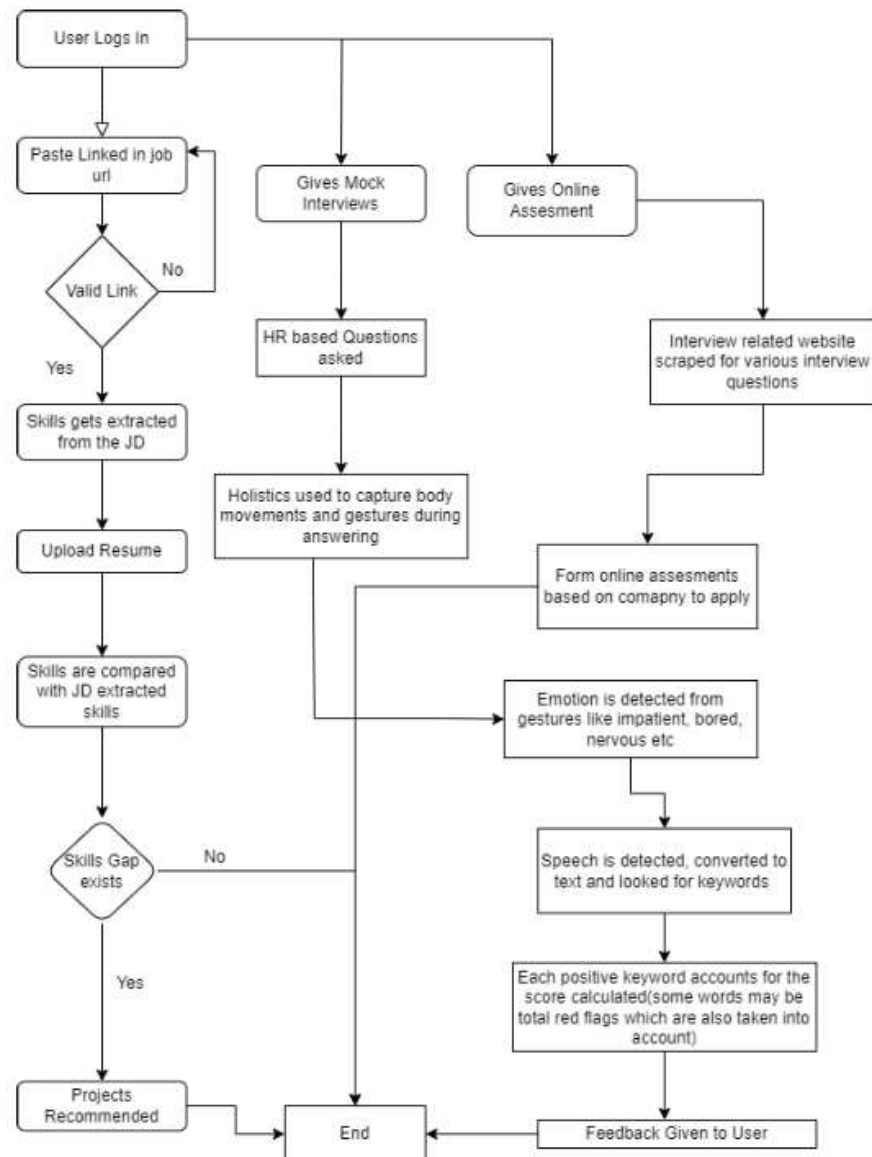


Fig. 1: System diagram

4 Implementation

The project consists of 3 modules:

4.1 RESUME ANALYSIS AND IMPROVEMENT

In the first module, the platform incorporates resume parsing and skill gap analysis. Users can find any job on LinkedIn by pasting its Uniform Resource Locator (URL). The job details, including the job description, can be retrieved through two methods:

a) Utilizing Selenium to automate the scraping process. This involves navigating to the LinkedIn page, identifying target Document Object Model (DOM) elements, copying their XML Path Language (XPath) from the browser's inspect element, and forming a JavaScript Object Notation (JSON) object with the necessary fields.

b) Leveraging the ProxyCurl LinkedIn Application Programming Interface (API) to fetch all required information directly from the LinkedIn job URL.

To determine the essential skills for the job, the platform employs NLP techniques. It utilizes the transformers summarization pipeline to extract the crux of the job description and employs KeyBERT to extract crucial keywords. This approach facilitates a comprehensive analysis of job requirements and aids users in understanding the skills needed for specific positions.

Note: Use MMR=true and a diversity of 0.4 is the preferred setting for the process.

```
@app.route('/search-company', methods=['POST'])
def getCompany():

    api_endpoint = 'https://nubela.co/proxycurl/api/linkedin/company'

    header_dic = {'Authorization': 'Bearer ' + api_key}
    params = {
        'url': request.form['url'],
        'resolve_numeric_id': 'true',
        'categories': 'include',
        'funding_data': 'include',
        'extra': 'include',
        'exit_data': 'include',
        'acquisitions': 'include',
        'use_cache': 'if-present',
    }
    response = requests.get(api_endpoint,
                           params=params,
                           headers=header_dic)

    return response.json()
```

Fig. 2: Scraping API implementation

Resumes in PDF format are uploaded and stored in Firebase Storage. The platform then extracts the entire text from the URL and feeds it to the ChatGPT 3.5 API for skill extraction. To effectively prompt the AI, a carefully crafted message is constructed. By employing prompt engineering techniques, a prompt is designed to elicit responses in a format that is preferred for display on the frontend.

```

@app.route('/resume/parse', methods=['POST'])
def getSkills():
    remote_file = urlopen(request.form['url']).read()
    memory_file = io.BytesIO(remote_file)
    reader = PdfReader(memory_file)
    textreader = pages[0].extract_text()
    openai.api_key = "sk-AmyIj3pcEFCpKgsolLOdT3B1bKf3K7ECu7RHVj25mZpStg8"

    response = openai.ChatCompletion.create(
        model="gpt-3.5-turbo",
        messages=[
            {"role": "system", "content": "You are a recruiter"},
            {"role": "user", "content": f"Find all the one word technical skills considering {text} as a candidate resume and return a list of skills where each skill is seprated by a comma without any additional text before or after the response"}]
    )

    result = ''
    for choice in response.choices:
        result=result+choice.message.content

    return result

```

Fig. 3: Resume parsing prompt

The skills present in the resume and the skills required for the given job description are then compared to display the overlapping and non-overlapping areas of interest. The cosine similarity distance is found between the given lists of skills to find areas of overlap. This can also be done using prompt engineering by using the ChatGPT API to do the same with this prompt.

```

@app.route('/skills/overlap', methods=['POST'])
def getOverlap():
    openai.api_key = "sk-AmyIj3pcEFCpKgsolLOdT3B1bKf3K7ECu7RHVj25mZpStg8"
    print(skills)
    response = openai.ChatCompletion.create(
        model="gpt-3.5-turbo",
        messages=[
            {"role": "system", "content": "You are a recruiter"},
            {"role": "user", "content": f"Find one word tech skills which are found in both the {request.form['skills']} which is a list of skills mentioned in {request.form['jd']} which is the job description and return a list of atleast 10 tech skills where each skill is seprated by a comma and without any additional text before or after the list of {skills}"}]
    )

    result = ''
    for choice in response.choices:
        result=result+choice.message.content

    return result

```

Fig. 4: Skill overlap prompt generation

The non-overlapping skills are then utilized to generate project recommendations along with associated GitHub profiles. The ChatGPT API is prompted to provide the results in a format suitable for the frontend. This approach enables users to discover project ideas that align with their skill gaps, offering an opportunity to enhance their proficiency. By exploring the suggested GitHub projects and cloning them, users can understand real-world use cases and gain insights into implementation details. It is important to note that these projects are foundational, and users can further explore advanced projects by browsing the internet.

```

app.route('/projects', methods=['POST'])
def getProjects():
    openai.api_key = "sk-OLj2H7jqmQ317b6vJ1T3B1bkFJQsRAOK5h9cBYjAPUATDY"
    response = openai.ChatCompletion.create(
        model="gpt-3.5-turbo",
        messages=[
            {"role": "system", "content": "You are a recruiter"},
            {"role": "user", "content": "Recommend one project required to learn skills mentioned\nin {request.form['skills']} as a list of objects with the skill, project and\ngithub_url being the keys of the object and return a list of objects separated by\ncomma with no additional text before and after the list"}]
    )
    result = ''
    for choice in response.choices:
        result=result+choice.message.content
    return result

```

Fig. 5: Recommend project prompt generation

4.2 VIDEO INTERVIEW MODEL

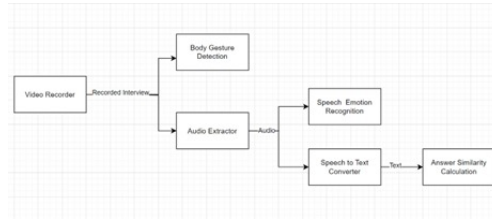


Fig. 6: Video interview model

1. Body Gesture Detection

This module detects the body gestures of the candidate during an interview. Any inappropriate gestures are then identified and the corresponding feedback is given to the user based on his overall body posture throughout the interview.

Dataset generation

MediaPipe's solution library incorporates a drawing utility that simplifies the process of visualizing landmarks, making it more efficient. The solution library's holistic model is then utilized to detect and record these landmarks for further processing. These landmarks are represented in terms of coordinates (x, y, z, v) and include face_landmarks, pose_landmarks, left_hand_landmarks, and right_hand_landmarks. In total, there are 501 coordinates for each landmark. A dataset containing 8000 such coordinate entries (rows) is created for this module by posing appropriately and manually labeling each gesture.

[illegible]

A few examples include:-

Fig. 8: Appropriate

Fig. 9: Disinterested

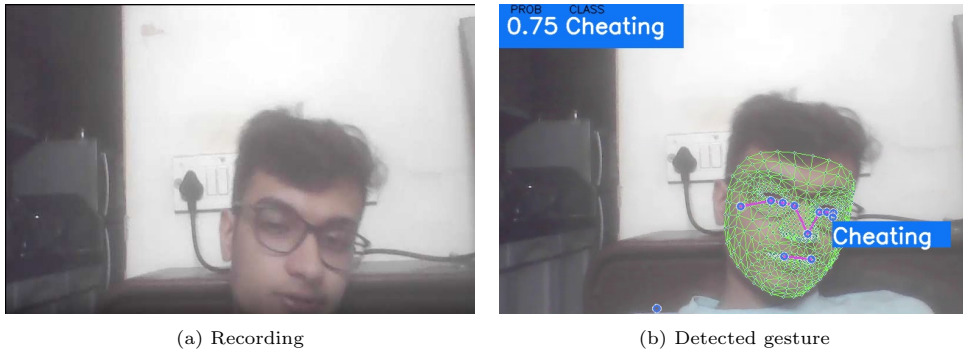


Fig. 10: Cheating



Fig. 11: Defensive

b) Through evaluations of the three models, the most accurate one is selected for further use.

c) Using the trained model, real-time detection of gestures is performed on unseen data. The results are rendered on the screen with the assistance of OpenCV.

d) The detected results, along with feedback, are stored and presented to the user. This feedback provides insights into the candidate's gestures and overall body posture during the interview, helping them understand and potentially improve their non-verbal communication skills.

- **Classification Algorithms**

Three algorithms were tested to categorize the coordinates into classes representing various body postures, including Appropriate, Disinterested, Confused, Cheating, Pride, and Defensive.

1. **Logistic Regression**

Logistic regression is a commonly employed supervised machine learning approach for classification problems. Its primary objective is to estimate the likelihood that a given instance belongs to a particular class. Utilizing a set of independent factors, logistic regression predicts the categorical dependent variable. The sigmoid function, also known as the logistic or sigmoid function, transforms expected values into probabilities, exhibiting an S-form curve. Logistic regression incorporates the concept of a threshold value, indicating the likelihood of either 0 or 1. In the context of body gesture classification, multinomial logistic regression is utilized to map coordinates to specific body gesture classes.

2. Ridge Classification

Ridge classification is a technique used in machine learning for analyzing linear discriminant models. It serves as a form of regularization aimed at preventing overfitting by penalizing model coefficients. This is achieved by introducing a penalty term into the cost function to discourage complexity. Typically, the penalty term consists of the sum of squared coefficients of the model's features, ensuring that the coefficients remain small and preventing overfitting.

3. Random forest

In the random forest technique, each decision tree within the ensemble is constructed from a bootstrap sample—data randomly selected from the training set with replacement. A portion, specifically one-third, of this training sample, is earmarked as test data, referred to as the out-of-bag (OOB) sample. To introduce additional randomization and enhance dataset variety while reducing decision tree correlation, feature bagging is employed. For a classification task, the projected class is determined through a majority vote, selecting the most common categorical variable. Lastly, the prediction is validated through cross-validation using the OOB sample.

2. Voice Based emotion recognition

- **LSTM model**

Long Short-Term Memory (LSTM) is a type of Recurrent Neural Network (RNN) that is specifically designed to process, predict, and classify based on sequential data such as time series, speech, and text. The model employed by this module consists of 7 layers - 1 LSTM, 3 Dropout, and 3 Dense Layers.

```

Model: "sequential_9"
=====
Layer (type)                 Output Shape              Param #
-----
lstm_10 (LSTM)               (None, 256)               264192
dropout_27 (Dropout)         (None, 256)               0
dense_27 (Dense)             (None, 128)               32896
dropout_28 (Dropout)         (None, 128)               0
dense_28 (Dense)             (None, 64)                8256
dropout_29 (Dropout)         (None, 64)                0
dense_29 (Dense)             (None, 7)                 455
=====
Total params: 305799 (1.17 MB)
Trainable params: 305799 (1.17 MB)
Non-trainable params: 0 (0.00 Byte)

```

Fig. 12: LSTM Output

40 audio features are extracted from the Toronto Emotional Speech Set (TESS) [9] consisting of 2800 audio samples distributed evenly across 7 cardinal emotions - anger, disgust, fear, happiness, pleasant surprise, sadness, and neutral. The LSTM model is trained on the extracted features, thereby producing a 99% training accuracy and 53.21% validation accuracy.

- **hcalabres/wav2vec2-lg-xlsr-en-speech-emotion-recognition model (pre-trained)**

This is a fine-tuned version of Wave2Vec 2.0 for a speech emotion recognition (SER) task constructed by Enrique Hernández Calabrés. It has been fine-tuned on the Ryerson Audio-Visual Database of Emotional Speech and Song (RAVDESS) [10] dataset consisting of 1440 samples of recordings from actors performing on 8 different emotions in English - angry, calm, disgust, fearful, happy, neutral, sad, and surprised. The model has an evaluation accuracy of 0.8223. Due to the superior accuracy of this model as compared to the LSTM model designed for this module, it is used to perform voice emotion recognition for this module.

3. Answer Evaluation

This module evaluates the user's response to questions during the interview. It includes converting the user's spoken answer to text, processing the response, and comparing it with the ideal answer to determine similarity levels. The goal is to generate feedback for the user based on this comparison.

3.1 Speech To Text

To convert the user's spoken answer to text, the process begins by recording the user's response and creating a Waveform Audio File Format (WAV) file, following which text extraction takes place. Once the answer is successfully transcribed, it is stored for subsequent processing.

3.2 Similarity Calculation

The next step involves processing the response to calculate the similarity between the ideal answer and the user's input. This includes **error checking** and **keyword matching**.

- **Word2Vec**

The word2vec technique learns word connections from a huge text corpus by using a neural network model. After being trained, this kind of model may identify related words or recommend terms to complete a sentence. Word2vec, as the name suggests, uses a specific vector—a list of numbers—to represent each unique word. Because the vectors are carefully selected to reflect the syntactic and semantic properties of words, the degree of semantic similarity between the words represented by those vectors may be determined using a straightforward mathematical function called cosine similarity. The source and target answers are provided to the pretrained model. It then generates the source vector representing the user answer and the target vector representing the target answer. By using the following formula the cosine similarity is calculated between the two vectors:-

$$csim = np.dot(vecA, vecB) / (np.linalg.norm(vecA) * np.linalg.norm(vecB))$$

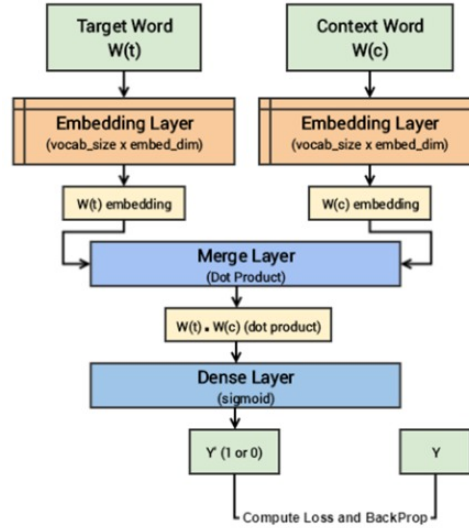


Fig. 13: Similarity between words

4.3 ONLINE ASSESSMENT GENERATION

The text is summarized using transformers’ summarization pipeline. Subsequently, KeyBERT is employed for keyword extraction. The context, comprising the summarized text, and keywords, representing the answers for that context, are provided to a question generation model. This model is constructed using transformers’ encoding-decoding architecture and is trained on the **SQuAD** [11] dataset. Currently, the model is trained on 8000 rows of the **SQuAD** dataset.

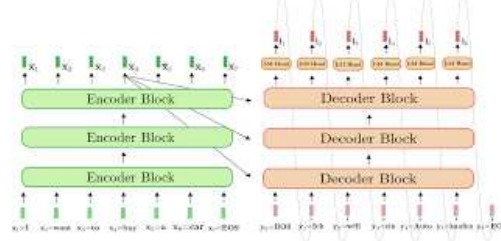


Fig. 14: Transformers encoding-decoding structure

A **Transformer** is a type of deep learning architecture that uses an attention mechanism to process text sequences. Unlike traditional models based on recurrent neural networks, Transformers do not rely on sequential connections and can capture long-term relationships in a text.

The Transformer operates based on two key components: attention and transformer blocks. Attention allows the model to assign different weights to words within a sequence and focus its attention on the most relevant parts of the text. Transformer blocks are layers that apply nonlinear transformations to input representations, aiding the model in learning language patterns and structures.

A structure is then formed, containing the context and the keyword, and questions are generated using beam search. The model currently operates with 5 beams for maximum optimization, ensuring both semantic and syntactic correctness.

Beam Search introduces two improvements over Greedy Search. Instead of selecting just the single best word at each position, Beam Search expands this approach by considering the best 'N' words. In the case of Greedy Search, each position is considered in isolation, focusing solely on the best word for that position without taking into account the preceding or following words. In contrast, Beam Search selects the 'N' best sequences, considering the probabilities of the combination of all preceding words along with the word in the current position. In essence, Beam Search casts the "light beam of its search" more broadly than Greedy Search, leading to its name. The hyperparameter 'N' is referred to as the Beam width.

Intuitively it makes sense that this yields better results over Greedy Search. Because, the thing of interest is the best complete sentence, which may be missed if only the best individual word in each position is picked.

```

@app.route('/api', methods=['POST'])
def getTest():
    openai.api_key = "sk-Amyi3pCEFCpkgsolOd38l0kF3K7ECu7RHVj25n2pPtQ8"

    response = openai.ChatCompletion.create(
        model="gpt-3.5-turbo",
        messages=[
            {"role": "system", "content": "You are an interviewer"},
            {"role": "user", "content": "Give me array of 10 mcq type interview question for "
            + (request.form['subject']) + " with (request.form['difficulty']) difficulty where each "
            + "question is a object with the following fields question,answers array of 4 elements, "
            + "correct_answer,correct_answer_indices which is a number ,only array is required with "
            + "no text before or after the array"))

    result = ""
    for choice in response.choices:
        result=result+choice.message.content

    return result

```

Fig. 15: Difficulty optimizing prompt

The file from which questions are generated is scraped in real-time from the Geeks for Geeks (GFG) page. This ensures that the data is consistently updated with the latest trends, allowing for the consideration of any new topics. The generated question is then fed into the ChatGPT API to produce distractors for the given question and its answer. Additionally, the difficulty of the question can be adjusted using the ChatGPT API, allowing for questions to be made easier or more advanced as desired. For each question, the user's answer is stored, and the score is calculated and saved for performance evaluation.

Each correct answer is marked with green, and incorrect answers are marked with red by mapping the user's answer to the correct known answer. There are also fallback JSON objects used for test generation if the ChatGPT server fails due to extensive usage or credit overuse.

The screenshot shows a web application titled "Interview-Ease". It has a header with "Home Subject: C++", "Selected Level: Intermediate", and a "Generate Test" button. The main content area displays a question: "Which data structure is best suited for implementing a priority queue?". Below the question are four radio button options: "Queue", "Linked List", "Tree", and "Heap". The "Queue" option is selected and highlighted with a green border. Below the options is a "Submit Answer" button. A large number "1" is visible on the right side of the question area. Below the question, there is a section for "After a few days complexity of algorithm in the next level?" with four radio button options: "O(n)", "O(n^2)", "O(n log n)", and "O(1)". The "O(n)" option is selected and highlighted with a green border. Below this section is a "Submit Answer" button.

Fig. 16: Beam search example

5 Results and Discussion

The home page consists of 3 major modules

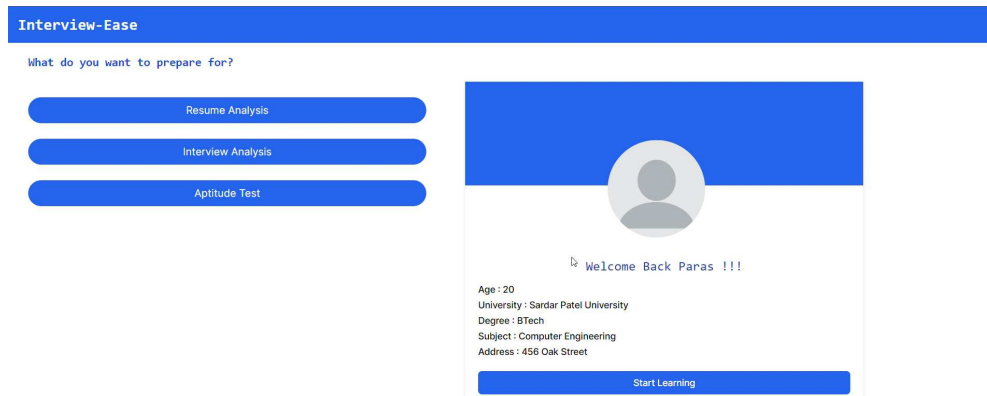


Fig. 17: Home page

5.1 Resume Parsing and Skill Gap Analysis

The URL is pasted in the input box and the resume is uploaded in Portable Document Format (PDF) format. The job details are scraped successfully, the resume is parsed and skills are fetched.

The "Skill analyze" button is pressed to check the percentage of overlapping and non-overlapping regions.

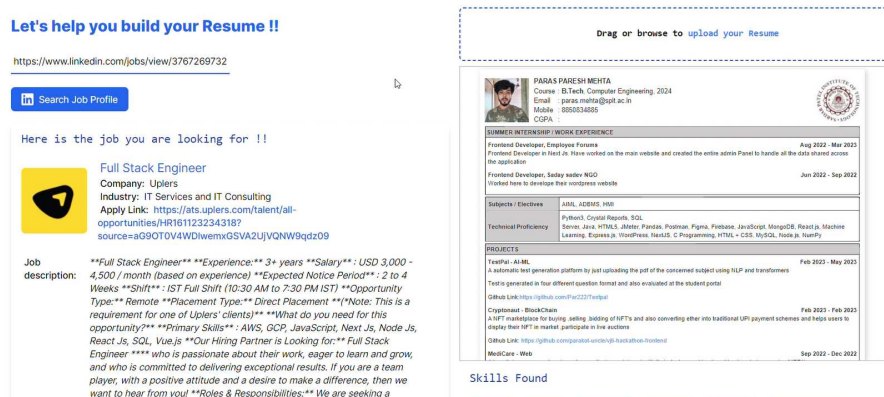


Fig. 18: Resume parsing

The non-overlapping regions also have project recommendations along with their GitHub URLs which redirect to the corresponding GitHub pages.

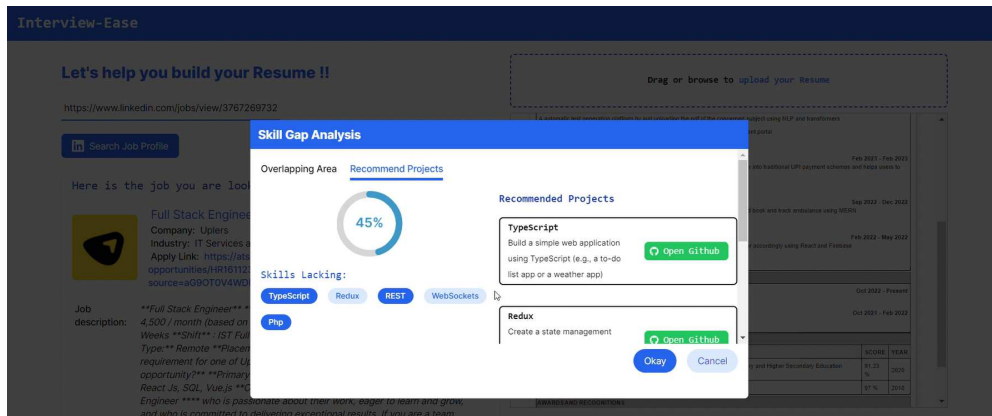


Fig. 19: Project Recommendation

5.2 Mock Interview Evaluation and Feedback Generation

Submodule	Accuracy
Body Gesture Detection	98%
Speech Emotion Recognition	82%
Answer Similarity Comparison	92%

Table 1: Submodule accuracies

- **Body Gesture detection classification algorithm results**
The following shows the accuracy of each model with the Ridge Classifier achieving the highest.

Logistic Regression	Ridge Classifier	Random Forest
0.9708	0.9825	0.9708

Table 2: Comparison of classification accuracies

Logistic Regression

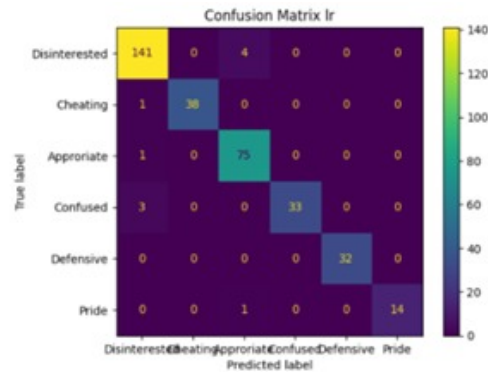


Fig. 20: Confusion Matrix

lr					
	precision	recall	f1-score	support	
Appropriate	0.97	0.97	0.97	145	
Cheating	1.00	0.97	0.99	39	
Confused	0.94	0.99	0.96	76	
Defensive	1.00	0.92	0.96	36	
Disinterested	1.00	1.00	1.00	32	
Pride	1.00	0.93	0.97	15	
accuracy			0.97	343	
macro avg	0.98	0.96	0.97	343	
weighted avg	0.97	0.97	0.97	343	

Fig. 21: Classification Report

Random forest

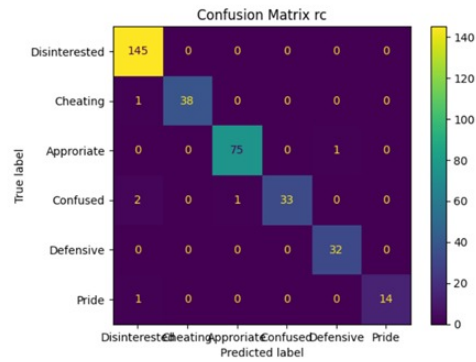


Fig. 22: Confusion Matrix

rc	precision	recall	f1-score	support
Appropriate	0.97	1.00	0.99	145
Cheating	1.00	0.97	0.99	39
Confused	0.99	0.99	0.99	76
Defensive	1.00	0.92	0.96	36
Disinterested	0.97	1.00	0.98	32
Pride	1.00	0.93	0.97	15
accuracy			0.98	343
macro avg	0.99	0.97	0.98	343
weighted avg	0.98	0.98	0.98	343

Fig. 23: Classification Report

Ridge Classifier

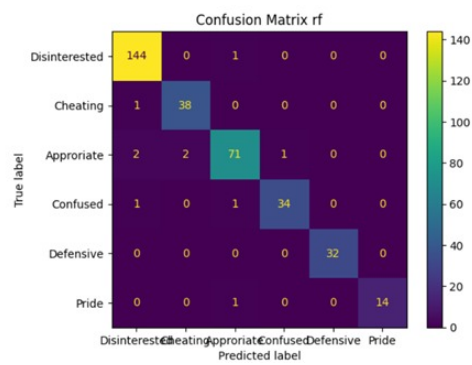


Fig. 24: Confusion Matrix

rf	precision	recall	f1-score	support
Appropriate	0.97	0.99	0.98	145
Cheating	0.95	0.97	0.96	39
Confused	0.96	0.93	0.95	76
Defensive	0.97	0.94	0.96	36
Disinterested	1.00	1.00	1.00	32
Pride	1.00	0.93	0.97	15
accuracy			0.97	343
macro avg	0.98	0.96	0.97	343
weighted avg	0.97	0.97	0.97	343

Fig. 25: Classification Report

The platform offers two categories of mock interviews - technical and human resources (HR).

The technical mock interviews consist of four subcategories - Java, Object Oriented Programming, Operating Systems, and Database Management Systems.



Fig. 26: Technical Mock Interview Categories

Below is an example of what attempting a mock interview looks like (Java in this case). A question is presented to the user along with some guidelines he should keep in mind. He can also see his video preview. The timer above the preview goes off once the user starts recording by pressing the "Start Recording" button.

He can stop recording when he feels satisfied with his answer and submit his attempt. However, if time runs out before he has finished, the recorded video is automatically submitted for evaluation.

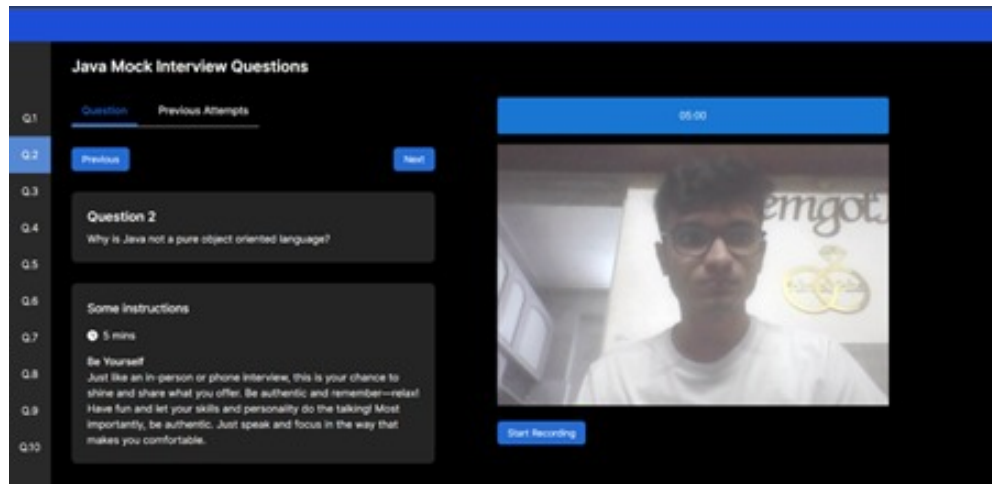


Fig. 27: Attempting the mock interview

The user can navigate back and forth between questions as well as switch tabs by clicking on the "Previous Attempts" option in order to have a look at his previous tries for a given question. The user can click the attempt he wishes to view by clicking on the corresponding option in the attempts history section.

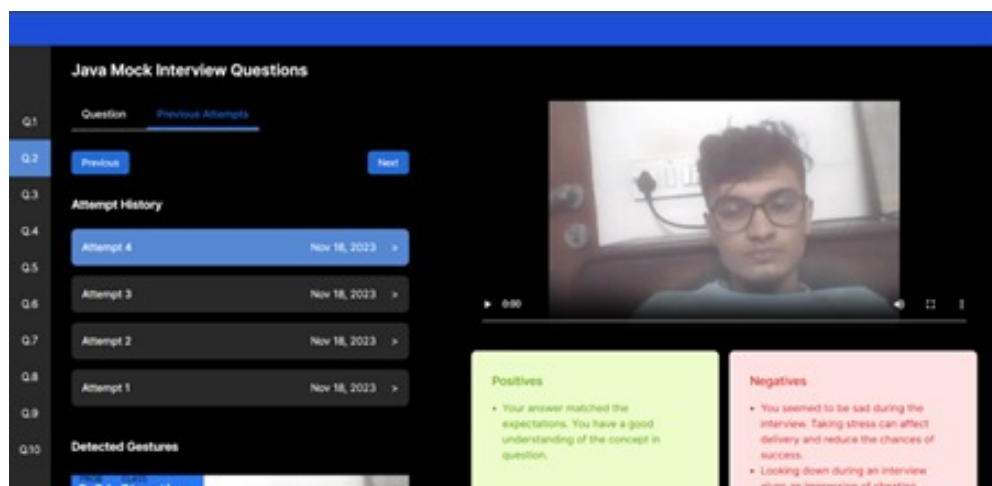


Fig. 28: Answer evaluation

Here, the user can view the submitted recording. He can also see the gestures that were detected at each instant during his interview and also some screenshots of

what each gesture looks like along with the probability of detection. The answer given by the user is evaluated against the answer expected for the question. Finally, he is given feedback on positives and negatives based on the correctness of the answer, the gestures detected as well as the emotion detected in his voice.

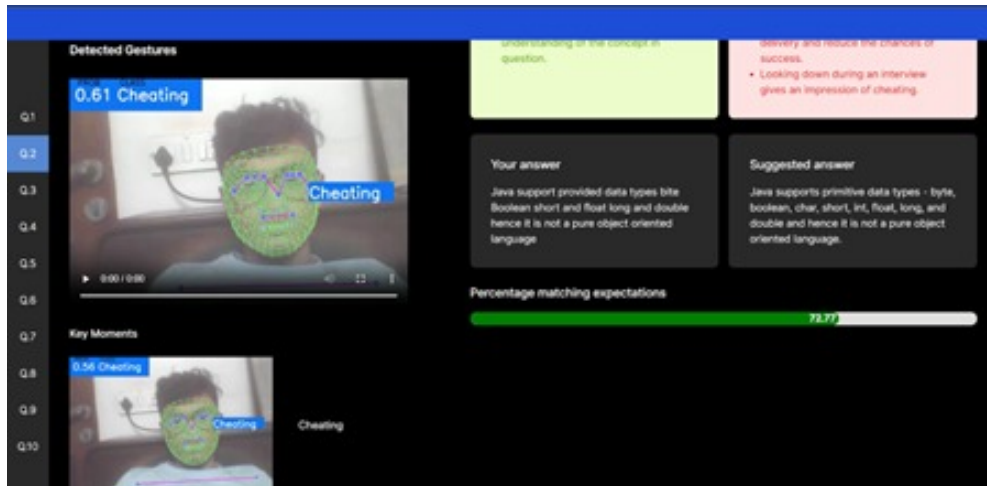


Fig. 29: Gestures are detected and feedback is shown

5.3 Aptitude test Analysis

- **Testing aptitude results**

The figure below shows a form which asks for the subject and the level of difficulty to generate a test.

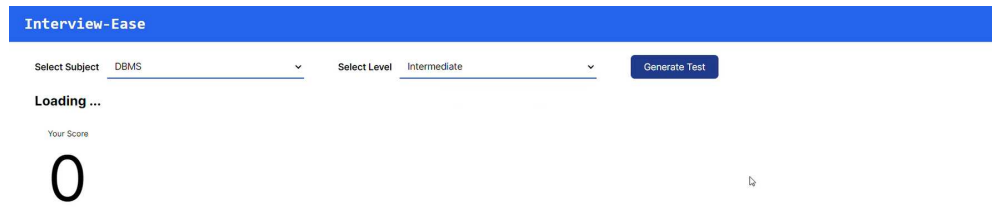


Fig. 30: Test Generation loading phase

Shown below is an example of how a DSA test of easy level can be generated and then how ChatGPT is used to enhance the difficulty level of the question generation model.

The questions generated are straightforward theory-based multiple-choice questions (MCQs). The user selects an option, and upon pressing the 'Check Answer' button, the chosen option is marked as green if it is the correct answer. The score is then maintained to keep track of the number of questions answered correctly by the user.

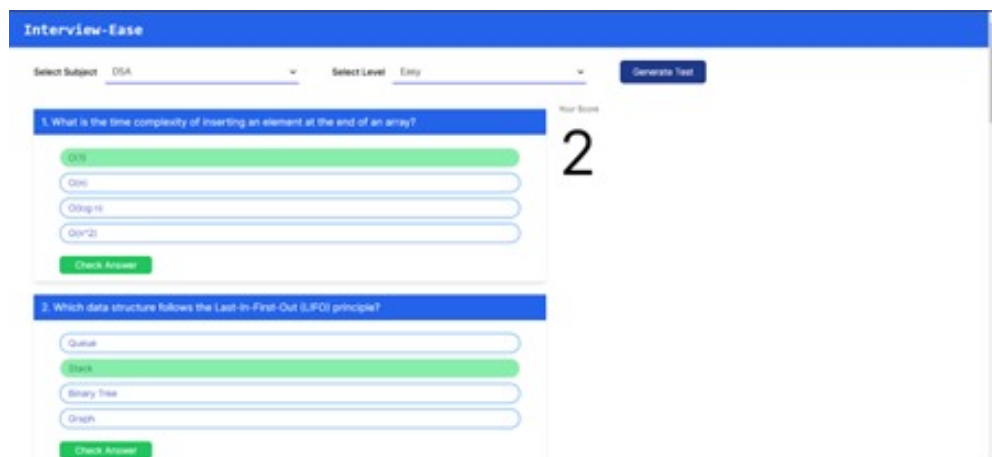


Fig. 31: Answering question correctly and increment score

The next figure shows how the ChatGPT 3.5 API transforms the difficulty levels of questions. In the following example a quiz of difficulty level - advanced is generated.



Fig. 32: Incorrect answer and display of correct option

5.4 Comparison with existing systems

Platforms	Features	Comparison and Shortcomings
Preplaced	Mock interviews with recruiters and industry professionals.	Limited availability of interviewers, high pricing, lacks resume analysis and quiz prep module.
Exponent	Interview preparation courses with live sessions and Q&A opportunities.	Limited free options, paid plans can be expensive. Questions might not be tailored to specific companies.
Interviewing.io	Detailed feedback and scorecard from industry experts.	Interviews are voice-only, lacking visual communication cues.
Pramp	Peer-to-peer practice with other job seekers through mutual mock interviews.	May lack expert feedback, no resume analysis, and no MCQ practice tests.
MockInterview.io	Flexible booking options and video recording/playback for review.	Limited availability of interviewers for specific roles or companies, not an anytime-anywhere experience, and can be costly.

Table 3: Comparison of Interview Preparation Platforms

6 Conclusion and Future Scope

While the application offers a comprehensive solution for automated placement preparation, several limitations should be addressed. The current speech emotion recognition model, hcalabres/wav2vec2-lg-xlsr-en, is optimized for 6-second audio samples, potentially impacting its accuracy for interviews lasting 1 to 1.5 minutes. A dedicated model trained on longer samples is essential for improved results.

Deciding that the interviewee is stressed solely based on any negative emotion detected (as is done currently) may not provide a precise measure. Exploring more concrete indicators for stress assessment is necessary for a more accurate evaluation as none currently exist. Developing a metric for evaluating interviewee confidence levels is also a potential area for improvement.

There are no well-defined right or wrong answers for HR questions, requiring manual intervention for assessment. This is a task that cannot be presently automated.

Future enhancements could include integrating a code editor and execution environment for technical skills evaluation, along with coding practice questions. Extending the platform to accommodate non-technical interviews is another avenue for expansion.

Further fine-tuning of the gesture detection model, exposure to a broader range of postures, and potential integration of a real coding interview experience can enhance the platform's capabilities, thereby providing a full-fledged interview experience.

References

- [1] Vinaya R. Kudatarkar, Manjula Ramannavar, Dr.Nandini S. Sidnal "An Unstructured Text Analytics Approach for Qualitative Evaluation of Resumes", 2015, IJIRAE.
- [2] Anonymous authors, "TOPIC-BASED QUESTION GENERATION", 2018, ICLR
- [3] Mengting Wan and Julian McAuley. "Modeling Ambiguity, Subjectivity, and Diverging Viewpoints in Opinion Question Answering Systems." 2016 IEEE 16th International Conference on Data Mining (ICDM), pp. 489–498, 2016.
- [4] Diederik P. Kingma and Jimmy Ba. "Adam: A Method for Stochastic Optimization." 3rd International Conference for Learning Representations, San Diego, 2015.
- [5] Joko Siswanto, Sinung Suakanto, Made Andriani, Margareta Hardiyanti, Tien Fabrianti Kusumasari, "Interview Bot Development with Natural Language Processing and Machine Learning", 2021, IJTech.

- [6] Abhimanyu Sherawat, Mayesh Mulay, Nihar Gupta, Supriya Anand, "Personality Recognition and Video Interview Analysis", 2021, IJERT.
- [7] Yosef Solewicz, Chagay Orenstein, Avital Friedland, "Predicting interviewee attitude and body language from speech descriptors", 2017, IJERT
- [8] Y. Li, 'Deep Learning of Human Emotion Recognition in Videos', Dissertation, 2017.
- [9] Pichora-Fuller, M. Kathleen; Dupuis, Kate, "Toronto emotional speech set (TESS)", Borealis, V1, 2020.
- [10] S. R. Livingstone, and F. A. Russo, "The Ryerson Audio-Visual Database of Emotional Speech and Song (RAVDESS): A dynamic, multimodal set of facial and vocal expressions in North American English." *PLOS ONE*, 13(5), e0196391, 2018.
IEEE Transactions on Industry Applications, vol. 45, no. 5, pp. 691-702, Oct. 1998.
- [11] P. Rajpurkar, J. Zhang, K. Lopyrev, and P. Liang, "SQuAD: 100,000+ Questions for Machine Comprehension of Text," arXiv:1606.05250 [cs.CL], To appear in Proceedings of the 2016 Conference on Empirical Methods in Natural Language Processing (EMNLP), June 2016.
- [12] M. Hansen, A. Pomp, K. Erki, and T. Meisen, "Data-Driven Recognition and Extraction of PDF Document Elements" Multidisciplinary Digital Publishing Institute, Vol.7, issue 3, September 2019.
- [13] Huang, ZY., Chiang, CC., Chen, JH. et al. A study on computer vision for facial emotion recognition. Sci Rep 13, 8425 (2023).
- [14] R.C. Chen, C. Dewi, S.W. Huang, and R.E. Caraka "Selecting critical features for data classification based on machine learning methods", Journal of Big Data, Article number: 52, July 2020.
- [15] Shubham Bhor, Vivek Gupta, Vishak Nair, Harish Shinde, "Resume Parser Using Natural Language Processing Techniques", 2021, IJRES.

7 Ethical Declarations

7.1 Conflict of Interest

The authors have no conflicts of interest to declare. The co-authors have seen and agree with the contents of the manuscript and there is no financial interest to report.